

PRD-03

Coding Practice Platform

Version 1.0

Objective

Develop a scalable coding practice backend that enables students to solve programming problems, submit solutions, run code against test cases, track performance, participate in daily challenges, and monitor coding progress.

The architecture should support future integration with external code execution services (Judge0, Piston, Docker sandbox, etc.) without requiring major changes.

Scope

This PRD includes:

- Problem Categories
 - Coding Problems
 - Test Cases
 - Starter Code Templates
 - Code Submission
 - Submission History
 - Execution Results
 - Daily Challenges
 - Coding Statistics
 - Difficulty Filtering
 - Tags
 - Company-wise Problems
 - Problem Discussions (basic)
 - Favorites
-

New Database Models

ProblemCategory

Examples:

- Arrays
- Strings
- Linked List
- Stack
- Queue
- Trees
- Graph
- Dynamic Programming
- Greedy
- Backtracking
- Bit Manipulation

Fields:

- id
 - name
 - slug
 - description
 - displayOrder
 - isActive
 - createdAt
 - updatedAt
-

CodingProblem

Fields:

- id
- categoryId
- title
- slug
- description (Markdown)
- problemStatement
- inputFormat
- outputFormat
- constraints
- sampleInput
- sampleOutput
- explanation
- difficulty (Easy/Medium/Hard)
- timeLimit
- memoryLimit
- acceptanceRate

- points
 - isPublished
 - createdAt
 - updatedAt
-

ProblemTag

Examples:

- Binary Search
- Two Pointer
- DFS
- BFS
- Sliding Window

Fields:

- id
 - name
 - slug
-

ProblemTagRelation

Many-to-many relationship between problems and tags.

Fields:

- id
 - problemId
 - tagId
-

Company

Examples:

- Google
- Amazon
- Microsoft
- Adobe
- Atlassian
- TCS
- Infosys

Fields:

- id
 - name
 - slug
 - logo
 - website
-

ProblemCompany

Many-to-many relationship.

Fields:

- id
 - problemId
 - companyId
-

TestCase

Fields:

- id
 - problemId
 - input
 - expectedOutput
 - isSample
 - isHidden
 - weight
-

CodeTemplate

Starter code.

Fields:

- id
- problemId
- language
- template

Supported languages:

- C
- C++
- Java

- Python
 - JavaScript
-

Submission

Fields:

- id
 - userId
 - problemId
 - language
 - sourceCode
 - status
 - runtime
 - memoryUsed
 - score
 - passedTestCases
 - totalTestCases
 - submittedAt
-

FavoriteProblem

Fields:

- id
 - userId
 - problemId
-

DailyChallenge

Fields:

- id
 - problemId
 - challengeDate
 - bonusXP
-

ProblemDiscussion

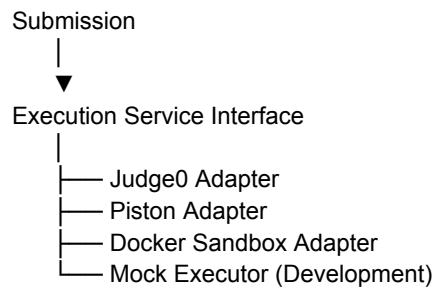
Fields:

- id
 - userId
 - problemId
 - content
 - createdAt
-

Future-ready Design

Do **not** execute code inside this backend.

Instead, design an abstraction:



Initially implement a **Mock Executor** so the architecture is ready for a real execution engine later.

APIs

Categories

GET /api/problem-categories
GET /api/problem-categories/:id
POST /api/problem-categories
PUT /api/problem-categories/:id
DELETE /api/problem-categories/:id

Problems

GET /api/problems
GET /api/problems/:id
POST /api/problems
PUT /api/problems/:id
DELETE /api/problems/:id

Filters:

- difficulty
- category
- tag
- company
- solved
- unsolved
- search

Support pagination and sorting.

Test Cases

GET /api/problems/:id/testcases

POST /api/testcases

PUT /api/testcases/:id

DELETE /api/testcases/:id

Hidden test cases should only be visible to admins.

Code Templates

GET /api/problems/:id/templates

POST /api/templates

PUT /api/templates/:id

DELETE /api/templates/:id

Submissions

POST /api/submissions

GET /api/submissions

GET /api/submissions/:id

GET /api/problems/:id/submissions

Favorites

POST /api/problems/:id/favorite

DELETE /api/problems/:id/favorite

GET /api/favorites

Daily Challenge

GET /api/daily-challenge

Returns today's challenge with metadata.

Discussions

GET /api/problems/:id/discussions

POST /api/problems/:id/discussions

PUT /api/discussions/:id

DELETE /api/discussions/:id

Statistics APIs

GET /api/stats/coding

Return:

- Total Solved
 - Easy Solved
 - Medium Solved
 - Hard Solved
 - Acceptance Rate
 - Total Submissions
 - Average Runtime
 - Current Streak
 - Longest Streak
 - Favorite Language
-

Search

Support searching by:

- Problem Title
- Tags
- Companies
- Category

- Difficulty
-

Business Rules

- Hidden test cases are never exposed to students.
 - Only admins can create/edit/delete problems.
 - Students can only edit/delete their own discussions.
 - Favorite entries must be unique per user/problem.
 - One daily challenge per date.
 - Code templates are mandatory for every supported language.
 - Slugs must remain unique.
-

Validation

Use Zod for:

- Difficulty enum
 - Positive time limits
 - Positive memory limits
 - Markdown description length
 - Valid programming language values
 - Unique slugs
 - Company website URL validation
-

Authorization

Student:

- Solve problems
- Submit code
- Favorite problems
- View discussions
- Post discussions

Mentor:

- Student permissions
- Moderate discussions

Admin:

- Full CRUD
 - Manage problems
 - Manage companies
 - Manage daily challenges
 - Manage tags
 - Manage categories
-

Performance Requirements

- Paginate all list endpoints.
 - Index:
 - categoryId
 - companyId
 - difficulty
 - slug
 - submittedAt
 - userId
 - Avoid N+1 queries.
 - Optimize relation loading with Prisma.
-

API Response Standard

Continue using the standardized response structure from PRD-01.

Acceptance Criteria

PRD-03 is complete when:

- All new Prisma models and relationships are implemented.
- Migrations execute successfully.
- CRUD APIs for categories, problems, test cases, templates, companies, tags, and discussions are functional.
- Submission APIs store execution metadata using the mock execution service.
- Favorites, daily challenge, and coding statistics work correctly.
- Filtering, pagination, sorting, and search are implemented.
- Role-based authorization is enforced.
- Hidden test cases remain inaccessible to students.
- Swagger documentation is updated.
- Logging, validation, and error handling follow the existing architecture.

- The backend is prepared for seamless integration with a real code execution engine (e.g., Judge0) without changing the submission workflow.
-

One architectural improvement

Rather than storing only aggregate statistics (like solved counts), **derive statistics from the Submission table** whenever possible. This keeps the source of truth in one place and avoids inconsistencies. If performance becomes an issue at scale, you can later introduce cached aggregates or materialized views without changing the API contract. This approach makes the initial implementation simpler and more reliable.